DBMS LAB Assignment-4

Tables: Customer(Cust_id, Fname, Lname, Area, Phone) and Movie(Mv_no, Cust_id, Title, Star, Price).

1. Create the Customer and Movie tables

SQL

```
CREATE TABLE Customer (
  Cust_id VARCHAR(5) PRIMARY KEY,
  Fname   VARCHAR(20) NOT NULL,
  Lname   VARCHAR(20),
  Area    VARCHAR(5) NOT NULL,
  Phone   VARCHAR(10)
);
CREATE TABLE Movie (
  Mv_no   INT PRIMARY KEY,
  Cust_id VARCHAR(5),
  Title   VARCHAR(30) NOT NULL,
  Star    VARCHAR(5) NOT NULL,
  Price   INT CHECK (Price BETWEEN 100 AND 250),
  FOREIGN KEY (Cust_id) REFERENCES Customer(Cust_id)
);
INSERT INTO Customer VALUES
('A01','Ivan','Ross','SA','6125467'),
('A02','Vandana','Ray','MU','5560379'),
('A03','Pramada','Jauguste','DA','4560389'),
('A04','Basu','Navindi','BA','6125401'),
('A05','Ravi','Shridhar','NA',NULL),
('A06','Rukmini','Aiyer','GH','5125274');
INSERT INTO Movie VALUES
(1,'A02','Bloody','JC',181),
(2,'A04','The Firm','TC',200),
(3,'A01','Pretty Woman','RG',151),
(4,'A06','Home Alone','MC',150),
(5,'A05','The Fugitive','MF',200),
(6,'A03','Coma','MD',100),
(7,'A02','Dracula','GO',150),
(8,'A06','Quick Change','BM',100),
(9,'A03','Gone with the Wind','CB',200),
(10,'A05','Carry on Doctor','LP',100);
DESC Customer;
DESC Movie;
SELECT * FROM Customer;
SELECT * FROM Movie;
```

OUTPUT

```
SQL> CREATE TABLE Customer ( Cust_id VARCHAR(5) PRIMARY KEY, Fname VARCHAR(20) NOT NULL, L

Table created.
SQL> CREATE TABLE Movie ( Mv_no INT PRIMARY KEY, Cust_id VARCHAR(5), Title VARCHAR(30) NOT

Table created.
SQL> INSERT INTO Customer VALUES ('A01','Ivan','Ross','SA','6125467'), ('A02','Vandana','R

6 rows created.
SQL> INSERT INTO Movie VALUES (1,'A02','Bloody','JC',181), (2,'A04','The Firm','TC',200),

10 rows created.
SQL> DESC Customer;

+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Cust_id | varchar(5)  | NO   | PRI | NULL    |       |
| Fname   | varchar(20) | NO   |     | NULL    |       |
| Lname   | varchar(20) | YES  |     | NULL    |       |
| Area    | varchar(5)  | NO   |     | NULL    |       |
| Phone   | varchar(10) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
SQL> DESC Movie;

+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Mv_no   | int(11)     | NO   | PRI | NULL    |       |
| Cust_id | varchar(5)  | YES  | MUL | NULL    |       |
| Title   | varchar(30) | NO   |     | NULL    |       |
| Star    | varchar(5)  | NO   |     | NULL    |       |
| Price   | int(11)     | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
SQL> SELECT * FROM Customer;

+---------+---------+----------+------+---------+
| Cust_id | Fname   | Lname    | Area | Phone   |
+---------+---------+----------+------+---------+
| A01     | Ivan    | Ross     | SA   | 6125467 |
| A02     | Vandana | Ray      | MU   | 5560379 |
| A03     | Pramada | Jauguste | DA   | 4560389 |
| A04     | Basu    | Navindi  | BA   | 6125401 |
| A05     | Ravi    | Shridhar | NA   | NULL    |
| A06     | Rukmini | Aiyer    | GH   | 5125274 |
+---------+---------+----------+------+---------+
SQL> SELECT * FROM Movie;

+-------+---------+--------------------+------+-------+
| Mv_no | Cust_id | Title              | Star | Price |
+-------+---------+--------------------+------+-------+
|     1 | A02     | Bloody             | JC   |   181 |
|     2 | A04     | The Firm           | TC   |   200 |
|     3 | A01     | Pretty Woman       | RG   |   151 |
|     4 | A06     | Home Alone         | MC   |   150 |
|     5 | A05     | The Fugitive       | MF   |   200 |
|     6 | A03     | Coma               | MD   |   100 |
|     7 | A02     | Dracula            | GO   |   150 |
|     8 | A06     | Quick Change       | BM   |   100 |
|     9 | A03     | Gone with the Wind | CB   |   200 |
|    10 | A05     | Carry on Doctor    | LP   |   100 |
+-------+---------+--------------------+------+-------+
```

2. Entity integrity constraint

SQL

```
-- Condition 1: duplicate primary key
INSERT INTO Customer VALUES ('A01','Tom','Cruz','LA','1234567');
-- Condition 2: NULL primary key
INSERT INTO Customer VALUES (NULL,'Tom','Cruz','LA','1234567');
-- Condition 1: duplicate primary key
INSERT INTO Movie VALUES (1,'A01','Titanic','LD',150);
-- Condition 2: NULL primary key
INSERT INTO Movie VALUES (NULL,'A01','Titanic','LD',150);
```

OUTPUT

```
SQL> INSERT INTO Customer VALUES ('A01','Tom','Cruz','LA','1234567');
ERROR 1062 (23000) at line 2: Duplicate entry 'A01' for key 'PRIMARY'
SQL> INSERT INTO Customer VALUES (NULL,'Tom','Cruz','LA','1234567');
ERROR 1048 (23000) at line 2: Column 'Cust_id' cannot be null
SQL> INSERT INTO Movie VALUES (1,'A01','Titanic','LD',150);
ERROR 1062 (23000) at line 2: Duplicate entry '1' for key 'PRIMARY'
SQL> INSERT INTO Movie VALUES (NULL,'A01','Titanic','LD',150);
ERROR 1048 (23000) at line 2: Column 'Mv_no' cannot be null
```

Result: Both tables reject a duplicate primary key and a NULL primary key, so entity integrity is ensured.

3. Referential integrity constraint

SQL

```
-- Child table: insert a Movie for a non-existing customer
INSERT INTO Movie VALUES (11,'A09','Titanic','LD',150);
-- Parent table: delete a customer who is referenced in Movie
DELETE FROM Customer WHERE Cust_id='A02';
```

OUTPUT

```
SQL> INSERT INTO Movie VALUES (11,'A09','Titanic','LD',150);
ERROR 1452 (23000) at line 2: Cannot add or update a child row: a foreign key constraint fails
    (`lab`.`Movie`, CONSTRAINT `Movie_ibfk_1` FOREIGN KEY (`Cust_id`) REFERENCES `Customer` (`C
    ust_id`))
SQL> DELETE FROM Customer WHERE Cust_id='A02';
ERROR 1451 (23000) at line 2: Cannot delete or update a parent row: a foreign key constraint fa
    ils (`lab`.`Movie`, CONSTRAINT `Movie_ibfk_1` FOREIGN KEY (`Cust_id`) REFERENCES `Customer`
     (`Cust_id`))
```

Result: A child row with a non-existent parent key is rejected, and a referenced parent row cannot be deleted, so referential integrity is ensured.

4. Domain integrity constraint (Movie)

SQL

```
-- Price outside 100..250 (too low)
INSERT INTO Movie VALUES (11,'A01','Titanic','LD',90);
-- Price outside 100..250 (too high)
INSERT INTO Movie VALUES (11,'A01','Titanic','LD',300);
-- NULL in NOT NULL column Title
INSERT INTO Movie VALUES (11,'A01',NULL,'LD',150);
-- NULL in NOT NULL column Star
INSERT INTO Movie VALUES (11,'A01','Titanic',NULL,150);
```

OUTPUT

```
SQL> INSERT INTO Movie VALUES (11,'A01','Titanic','LD',90);
ERROR 4025 (23000) at line 2: CONSTRAINT `Movie.Price` failed for `lab`.`Movie`
SQL> INSERT INTO Movie VALUES (11,'A01','Titanic','LD',300);
ERROR 4025 (23000) at line 2: CONSTRAINT `Movie.Price` failed for `lab`.`Movie`
SQL> INSERT INTO Movie VALUES (11,'A01',NULL,'LD',150);
ERROR 1048 (23000) at line 2: Column 'Title' cannot be null
SQL> INSERT INTO Movie VALUES (11,'A01','Titanic',NULL,150);
ERROR 1048 (23000) at line 2: Column 'Star' cannot be null
```

Result: The CHECK (Price between 100 and 250) and NOT NULL constraints reject invalid values, so domain integrity is ensured.

5. Movie titles with price between 100 and 200

SQL

```
SELECT Title FROM Movie WHERE Price > 100 AND Price < 200;
```

OUTPUT

```
SQL> SELECT Title FROM Movie WHERE Price > 100 AND Price < 200;

+--------------+
| Title        |
+--------------+
| Bloody       |
| Pretty Woman |
| Home Alone   |
| Dracula      |
+--------------+
```

6. Customers who saw movies with stars JC, TC or MC

SQL

```
SELECT Cust_id FROM Movie WHERE Star IN ('JC','TC','MC');
```

OUTPUT

```
SQL> SELECT Cust_id FROM Movie WHERE Star IN ('JC','TC','MC');

+---------+
| Cust_id |
+---------+
| A02     |
| A04     |
| A06     |
+---------+
```

7. Customers with an A in area name

SQL

```
SELECT * FROM Customer WHERE Area LIKE '%A%';
```

OUTPUT

```
SQL> SELECT * FROM Customer WHERE Area LIKE '%A%';

+---------+---------+----------+------+---------+
| Cust_id | Fname   | Lname    | Area | Phone   |
+---------+---------+----------+------+---------+
| A01     | Ivan    | Ross     | SA   | 6125467 |
| A03     | Pramada | Jauguste | DA   | 4560389 |
| A04     | Basu    | Navindi  | BA   | 6125401 |
| A05     | Ravi    | Shridhar | NA   | NULL    |
+---------+---------+----------+------+---------+
```

8. Titles with price within 180 and exactly 6 characters

SQL

```
SELECT Title FROM Movie WHERE Price <= 180 AND Title LIKE '______';
-- Check: 'Bloody' is the only 6-letter title and its price is 181 (>180)
SELECT Title, Price FROM Movie WHERE Title LIKE '______';
```

OUTPUT

```
SQL> SELECT Title FROM Movie WHERE Price <= 180 AND Title LIKE '______';

no rows selected
SQL> SELECT Title, Price FROM Movie WHERE Title LIKE '______';

+--------+-------+
| Title  | Price |
+--------+-------+
| Bloody |   181 |
+--------+-------+
```

Result: Only 'Bloody' has 6 characters, but its price is 181 (more than 180), so no rows are selected.

9. Price after 10% increment

SQL

```
SELECT Title, Price, Price * 1.10 AS Incremented_Price FROM Movie;
```

OUTPUT

```
SQL> SELECT Title, Price, Price * 1.10 AS Incremented_Price FROM Movie;

+--------------------+-------+-------------------+
| Title              | Price | Incremented_Price |
+--------------------+-------+-------------------+
| Bloody             |   181 |            199.10 |
| The Firm           |   200 |            220.00 |
| Pretty Woman       |   151 |            166.10 |
| Home Alone         |   150 |            165.00 |
| The Fugitive       |   200 |            220.00 |
| Coma               |   100 |            110.00 |
| Dracula            |   150 |            165.00 |
| Quick Change       |   100 |            110.00 |
| Gone with the Wind |   200 |            220.00 |
| Carry on Doctor    |   100 |            110.00 |
+--------------------+-------+-------------------+
```

10. Customer details as a sentence

SQL

```
SELECT CONCAT(Fname,' ',Lname,' stays in ',Area,' and his phone number is ',Phone,'.') AS Detai
    ls FROM Customer WHERE Phone IS NOT NULL;
```

OUTPUT

```
SQL> SELECT CONCAT(Fname,' ',Lname,' stays in ',Area,' and his phone number is ',Phone,'.'

+-------------------------------------------------------------+
| Details                                                     |
+-------------------------------------------------------------+
| Ivan Ross stays in SA and his phone number is 6125467.      |
| Vandana Ray stays in MU and his phone number is 5560379.    |
| Pramada Jauguste stays in DA and his phone number is 4560389. |
| Basu Navindi stays in BA and his phone number is 6125401.   |
| Rukmini Aiyer stays in GH and his phone number is 5125274.  |
+-------------------------------------------------------------+
```

Result: Ravi Shridhar has a NULL phone number, so his row is excluded.

11. Add NOT NULL to Lname

SQL

```
ALTER TABLE Customer MODIFY Lname VARCHAR(20) NOT NULL;
DESC Customer;
```

OUTPUT

```
SQL> ALTER TABLE Customer MODIFY Lname VARCHAR(20) NOT NULL;

Table altered.
SQL> DESC Customer;

+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Cust_id | varchar(5)  | NO   | PRI | NULL    |       |
| Fname   | varchar(20) | NO   |     | NULL    |       |
| Lname   | varchar(20) | NO   |     | NULL    |       |
| Area    | varchar(5)  | NO   |     | NULL    |       |
| Phone   | varchar(10) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
```

12. Customer with no phone number

SQL

```
SELECT Fname, Lname FROM Customer WHERE Phone IS NULL;
```

OUTPUT

```
SQL> SELECT Fname, Lname FROM Customer WHERE Phone IS NULL;

+-------+----------+
| Fname | Lname    |
+-------+----------+
| Ravi  | Shridhar |
+-------+----------+
```

13. Add the missing phone number

SQL

```
UPDATE Customer SET Phone='9830012345' WHERE Cust_id='A05';
SELECT * FROM Customer WHERE Area LIKE '%A%';
```

OUTPUT

```
SQL> UPDATE Customer SET Phone='9830012345' WHERE Cust_id='A05';

1 row updated.
SQL> SELECT * FROM Customer WHERE Area LIKE '%A%';

+---------+---------+----------+------+------------+
| Cust_id | Fname   | Lname    | Area | Phone      |
+---------+---------+----------+------+------------+
| A01     | Ivan    | Ross     | SA   | 6125467    |
| A03     | Pramada | Jauguste | DA   | 4560389    |
| A04     | Basu    | Navindi  | BA   | 6125401    |
| A05     | Ravi    | Shridhar | NA   | 9830012345 |
+---------+---------+----------+------+------------+
```

14. Unique customer ids from Movie

SQL

```
SELECT DISTINCT Cust_id FROM Movie;
```

OUTPUT

```
SQL> SELECT DISTINCT Cust_id FROM Movie;

+---------+
| Cust_id |
+---------+
| A01     |
| A02     |
| A03     |
| A04     |
| A05     |
| A06     |
+---------+
```

15. Remove NOT NULL from Star

SQL

```
ALTER TABLE Movie MODIFY Star VARCHAR(5) NULL;
DESC Movie;
```

OUTPUT

```
SQL> ALTER TABLE Movie MODIFY Star VARCHAR(5) NULL;

Table altered.
SQL> DESC Movie;

+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| Mv_no   | int(11)     | NO   | PRI | NULL    |       |
| Cust_id | varchar(5)  | YES  | MUL | NULL    |       |
| Title   | varchar(30) | NO   |     | NULL    |       |
| Star    | varchar(5)  | YES  |     | NULL    |       |
| Price   | int(11)     | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
```

16. Delete a row from Customer

SQL

```
DELETE FROM Customer WHERE Cust_id='A01';
```

OUTPUT

```
SQL> DELETE FROM Customer WHERE Cust_id='A01';
ERROR 1451 (23000) at line 1: Cannot delete or update a parent row: a foreign key constraint fa
    ils (`lab`.`Movie`, CONSTRAINT `Movie_ibfk_1` FOREIGN KEY (`Cust_id`) REFERENCES `Customer`
     (`Cust_id`))
```

Result: Fails because Cust_id A01 is referenced by Movie.

17. Delete a row from Movie

SQL

```
DELETE FROM Movie WHERE Mv_no=10;
SELECT * FROM Movie;
```

OUTPUT

```
SQL> DELETE FROM Movie WHERE Mv_no=10;

1 row deleted.
SQL> SELECT * FROM Movie;

+-------+---------+--------------------+------+-------+
| Mv_no | Cust_id | Title              | Star | Price |
+-------+---------+--------------------+------+-------+
|     1 | A02     | Bloody             | JC   |   181 |
|     2 | A04     | The Firm           | TC   |   200 |
|     3 | A01     | Pretty Woman       | RG   |   151 |
|     4 | A06     | Home Alone         | MC   |   150 |
|     5 | A05     | The Fugitive       | MF   |   200 |
|     6 | A03     | Coma               | MD   |   100 |
|     7 | A02     | Dracula            | GO   |   150 |
|     8 | A06     | Quick Change       | BM   |   100 |
|     9 | A03     | Gone with the Wind | CB   |   200 |
+-------+---------+--------------------+------+-------+
```

18. Drop the Customer table

SQL

```
DROP TABLE Customer;
```

OUTPUT

```
SQL> DROP TABLE Customer;
ERROR 1451 (23000) at line 1: Cannot delete or update a parent row: a foreign key constraint fa
    ils
```

Result: Fails because Movie has a foreign key referring to Customer.

20. Drop the foreign key from Movie

SQL

```
ALTER TABLE Movie DROP FOREIGN KEY Movie_ibfk_1;
SHOW CREATE TABLE Movie\G
```

OUTPUT

```
SQL> ALTER TABLE Movie DROP FOREIGN KEY Movie_ibfk_1;

Table altered.
SQL> SHOW CREATE TABLE Movie

*************************** 1. row ***************************
       Table: Movie
Create Table: CREATE TABLE `Movie` (
  `Mv_no` int(11) NOT NULL,
  `Cust_id` varchar(5) DEFAULT NULL,
  `Title` varchar(30) NOT NULL,
  `Star` varchar(5) DEFAULT NULL,
  `Price` int(11) DEFAULT NULL CHECK (`Price` between 100 and 250),
  PRIMARY KEY (`Mv_no`),
  KEY `Cust_id` (`Cust_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

Result: Q20 is done before Q19 because the Movie table must still exist to drop its foreign key.

19. Drop the Movie table

SQL

```
DROP TABLE Movie;
SHOW TABLES;
```

OUTPUT

```
SQL> DROP TABLE Movie;

Table dropped.
SQL> SHOW TABLES;

+---------------+
| Tables_in_lab |
+---------------+
| Customer      |
+---------------+
```

Result: After Q20 the Movie table can be dropped; only Customer remains.